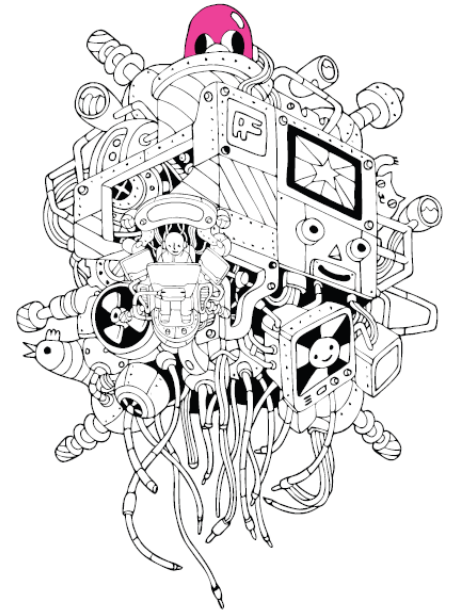


C programming



C Programming:

Chapter 09. Core of the C Language ! Functions !

C programming



Chapter 09-1. Defining& Declaring a function

C Programming:

Why create a function?

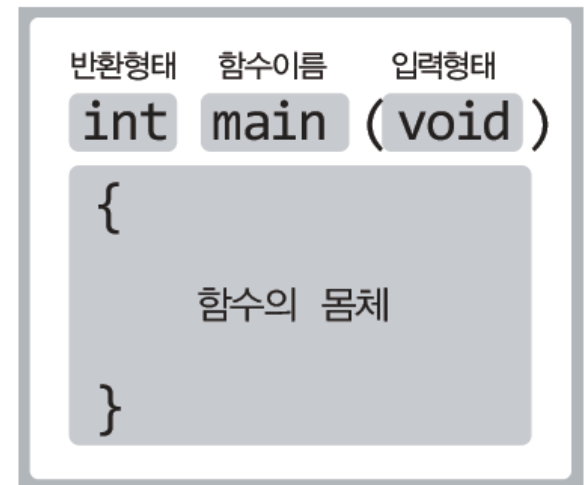
“Divide and Conquer!”

By writing a program by creating many small unit functions, you can achieve the effect of solving a large problem by breaking it down into smaller parts.

However, the reasons and benefits of creating functions are much more diverse.

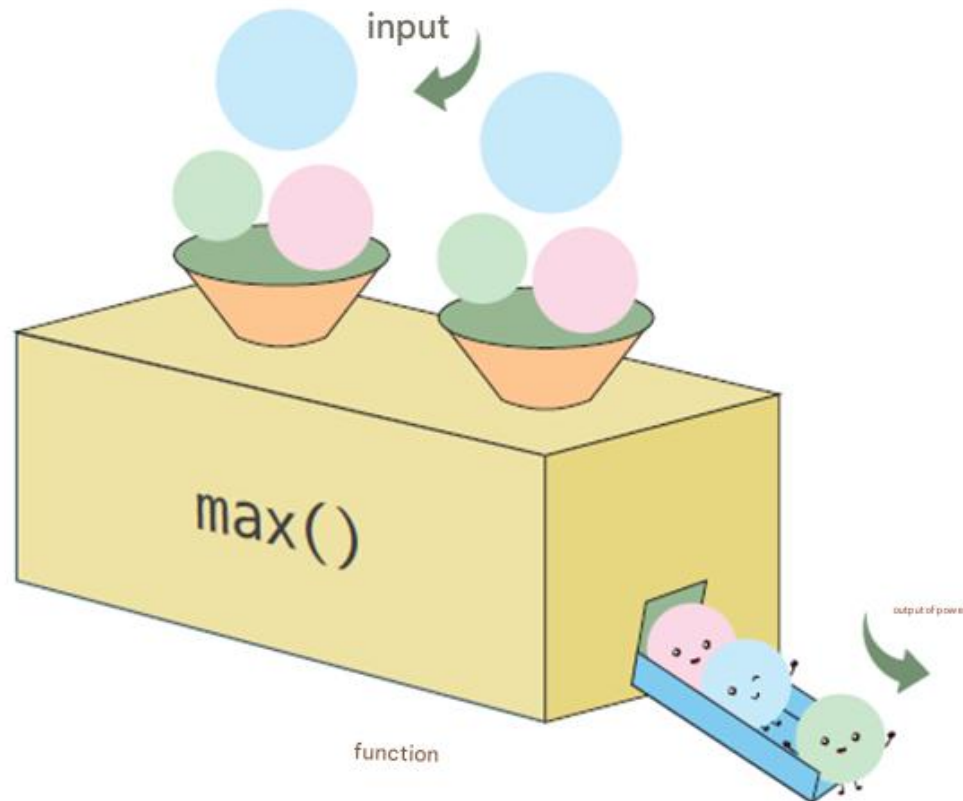
The smaller the function size, including the main function, the better. But smaller isn't always better, but must be taken to avoid creating unnecessarily large functions.

A function should be designed to do only one thing. Of course, what constitutes **one thing is a very subjective**. However, this subjective will also become clear as you gain experience in programming. It becomes a very clear standard .



Concept of function

- Function : It is like a black box that takes input, performs a specific task, and returns a result.



The function Why is it needed?

- The same code is used in multiple places .

I have similar code,
Can I combine them into one?



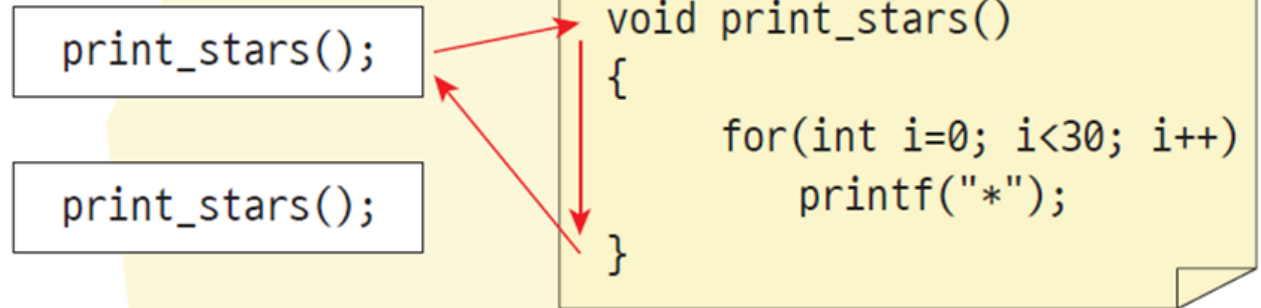
```
for(int i=0; i<30; i++)  
    printf("*");
```

```
for(int i=0; i<30; i++)  
    printf("*");
```

The function Why is it needed?

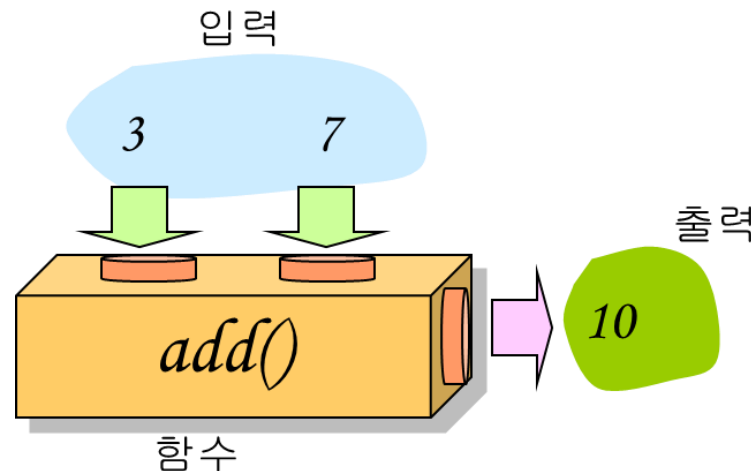
- By writing functions, you can create identical code in many place

Just use a function!



Features of functions

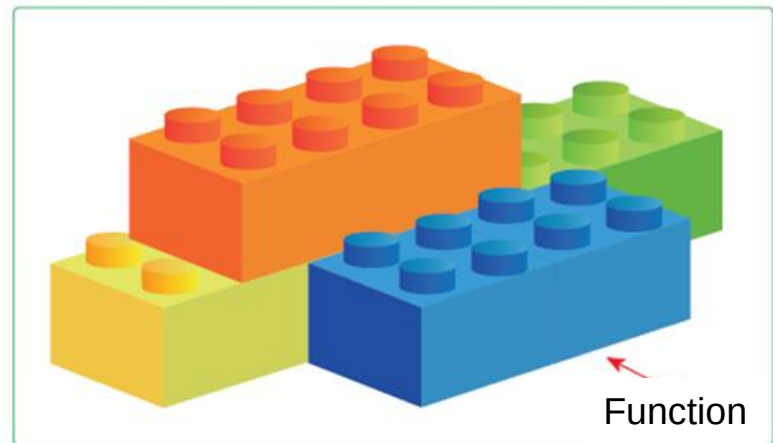
- A function is a collection of instructions to perform a specific task
- Functions have distinct names
- A function performs a specific task
- A function can take input and return a result



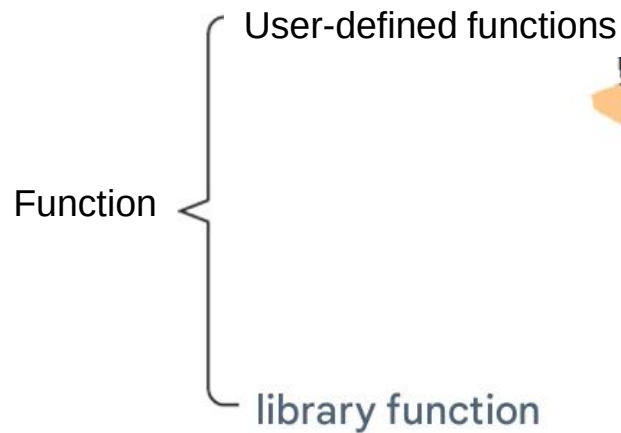
Pros of function

- Using functions can help **prevent duplication** of code .
- Once a function is written, it can be **reused** multiple times .
- Using functions allows you to divide your entire program into modules, making the **development process easier, more organized, and easier to maintain.**

Program



Types of functions



Since the function I want isn't available, I have to create it myself.



These functions are provided By default.

Definition of a function

Syntax

Function definition

Return type Function name Parameter (none)

`void print_stars()`

`{`

`for(int i=0; i<30; i++)
printf("*");`

Function body

`}`

Definition of a function

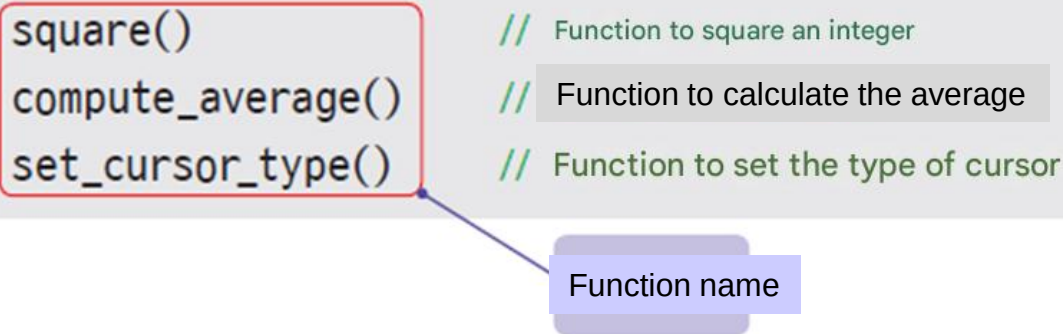
- Return type

.The return type refers to the type of data that a function returns to the caller after completing processing.

- Function name

- . A function name can be any name that follows the rules for identifiers .
- . It is a good idea to use (verb + noun) that implies the function's functionality.

```
int square()  
double compute_average()  
void set_cursor_type()
```



// Function to square an integer
// Function to calculate the average
// Function to set the type of cursor

Function name

Tip: Function's length



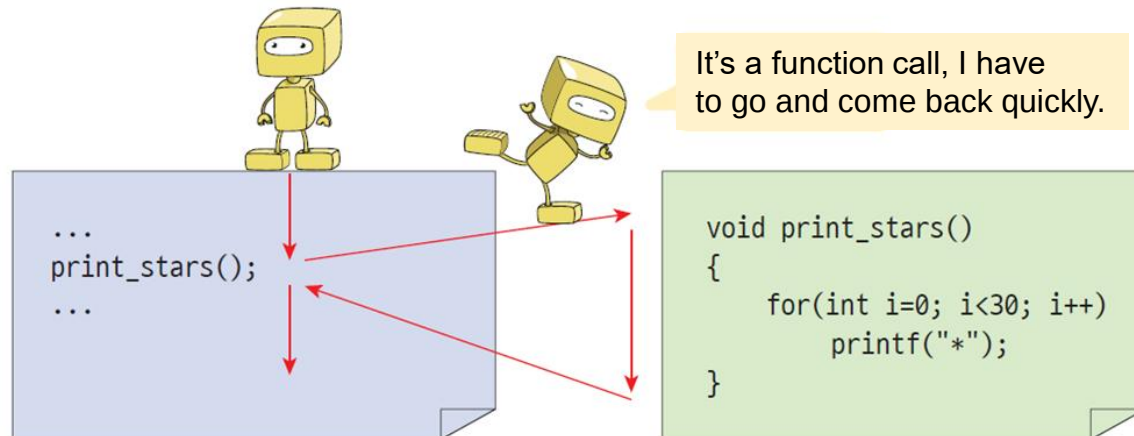
Is there a limit to the length of a function?

In C, there is no limit to the length of a function. This means that you can put many statements in a single function. However, **it is not good if the function length becomes too long**. Basically, **a function should perform only one task**. If the function length becomes too long, it should be considered as performing more than one task. Therefore, it is better to split the function. So, what is the appropriate length? Of course, there is no absolute standard, but it is good to keep it from exceeding 30 lines.

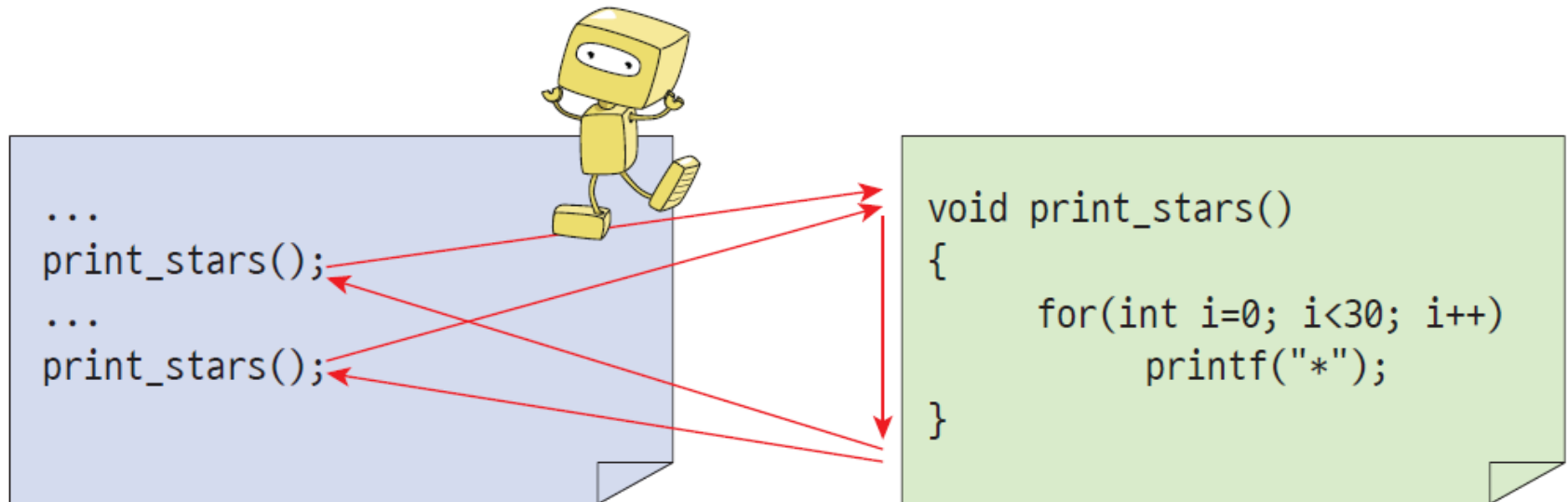


Function call

- A function call is writing the name of a function, such as `print_stars()` .
- The statements within a function are not executed at all until they are called. When a function is called, the currently executing code is suspended for a moment, and execution moves to the called function, where the statements within the function body are executed sequentially.
- When the execution of the called function is finished, it returns to the calling location and resumes execution of the code that was paused.



A function can be called multiple times



Input and output of functions : The printf function also returns .

```
int main(void)
{
    int num1, num2;
    num1=printf("12345\n");
    num2=printf("I love my home\n");
    printf("%d %d \n", num1, num2);
    return 0;
}
```

The printf function actually returns a value. It's just that the return value isn't needed, so it's not stored.

The printf function returns the length of the printed string.

```
12345
I love my home
6 15
```

Execution results

When a function returns a value, you can think of the returned value as replacing the function call statement. the printf function call below prints 6 If you return it,

```
num1=printf("12345\n");
```

The result of the function call is as follows, and the assignment operation is performed .

```
num1=6;
```



Function classification

Type 1: There are arguments and a return value! Arguments (○), return value (○)

Type 2: There are arguments, but no return value! Argument (○), return value (×)

Type 3: No arguments , but return value! Argument (×), return value (○)

Type 4: No arguments, no return value! Arguments (×), return value (×)

Distinction between functions based on the presence or
absence of arguments and return values!



If both the argument and return value are present

The arguments are two int type integers, and addition is performed using these two.

The result of addition is returned , so the return type is also declared as int.

Finally, let's name the function Add!

```
A. B. C.  
int Add (int num1, int num2)  
{  
    int result = num1 + num2;  
    D. return result;  
}
```



A. Return type
B. Function name
C. Parameter
D. Return of value

*Addition is performed first and
the result is returned.*

```
int Add(int num1, int num2)  
{  
    return num1+num2;  
}  
  
int main(void)  
{  
    int result;  
    result = Add(3, 4);  
    printf("Addition Result 1: %d \n", result);  
    result = Add(5, 8);  
    printf("Addition result 2: %d \n", result);  
    return 0;  
}
```

When the function call is completed, it returns to the location where it was called.

Execution results

Addition Result 1: 7

Addition Result 2: 13

If the argument or return value does not exist

```
void ShowAddResult(int num) // Pass arguments (0), return value (X)
{
    printf("Output addition result: %d \n", num);
}
```

```
int ReadNum(void) // Pass arguments (X), return value (0)
{
    int num;
    scanf("%d", &num);
    return num;
}
```

```
void HowToUseThisProg(void) // Passing arguments (X), return value (X)
{
    printf("Enter two integers to see the addition result. \n");
    printf("Okay! Now, please enter two integers.\n");
}
```



An example combining 4 function types

```
int Add(int num1, int num2)    // Pass arguments (0), return value (0)
{
    return num1+num2;
}

void ShowAddResult(int num)    // Pass arguments (0), return value (X)
{
    printf("Output addition result: %d \n", num);
}

int ReadNum(void)             // Pass arguments (X), return value (0)
{
    int num;
    scanf("%d", &num);
    return num;
}

void HowToUseThisProg(void)    // Passing arguments (X), return value (X)
{
    printf("Enter two integers to see the addition result. \n");
    printf("Okay! Now, please enter two integers.\n");
}
```

```
int main(void)
{
    int result, num1, num2;
    HowToUseThisProg();
    num1=ReadNum();
    num2=ReadNum();
    result = Add(num1, num2);
    ShowAddResult(result);
    return 0;
}
```



return that does not return a value

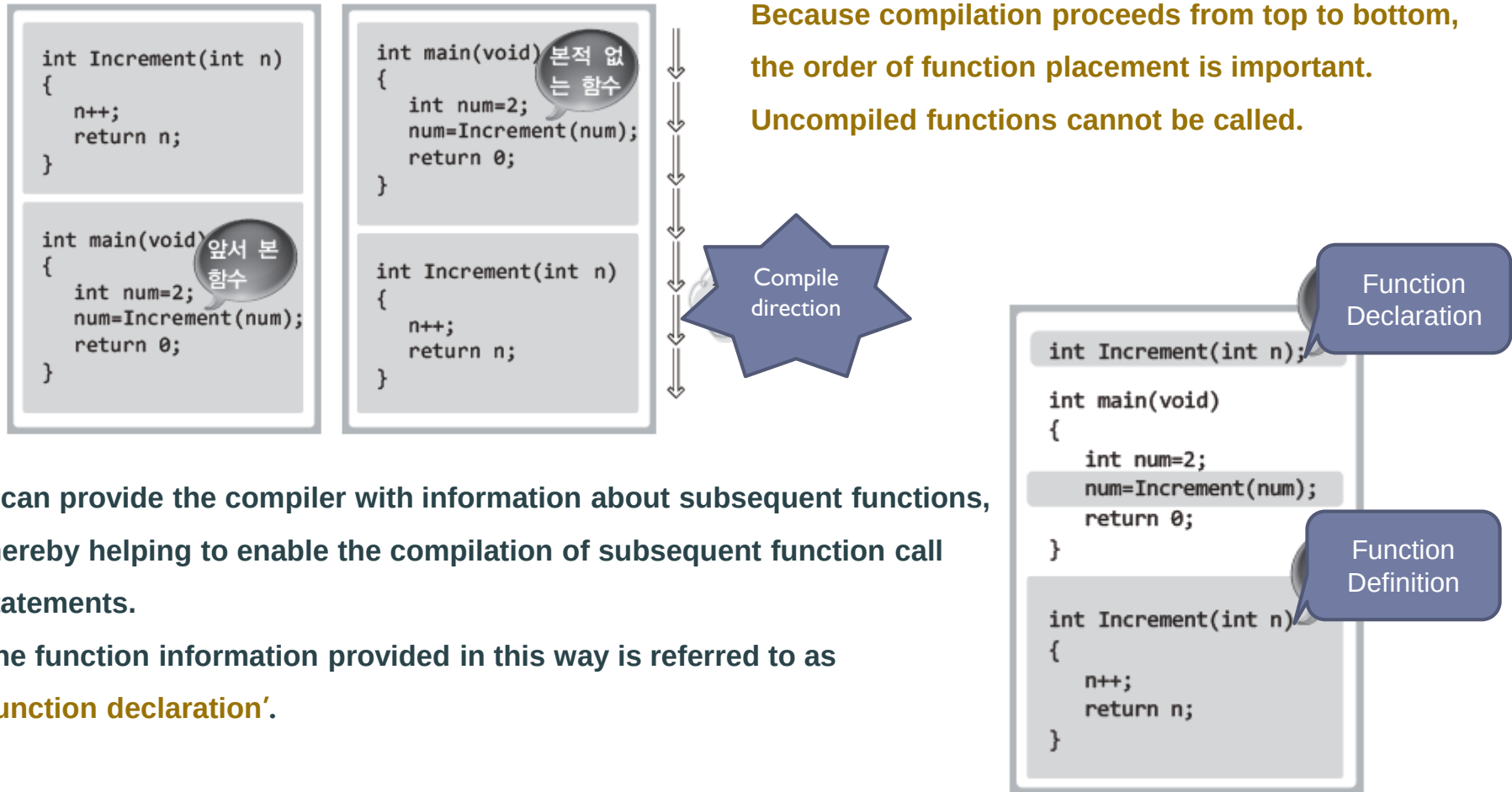
```
void NoReturnType(int num)
{
    if(num<0)
        return;    // A return statement that does not return a value!
    . . . .
}
```

The return statement has two meanings: 'returning a value' and 'exiting a function' .

As you can see in the example above, the return statement is structured in a way that does not return a value. It does not return a value, but can be used to exit a function.



Definition of a function and declaration of its prototype



It can provide the compiler with information about subsequent functions, thereby helping to enable the compilation of subsequent function call statements.

The function information provided in this way is referred to as **‘function declaration’**.

`int Increment(int n);` // function declaration

`int Increment(int);` // Same function declaration as above, parameter names can be omitted

Function prototype

- *Function prototyping* : Telling the compiler about a function in advance



```
#include <stdio.h>
double c_to_f ( double c_temp ); // Function prototype

int main( void )
{
    printf ( " Celsius is % lf and Fahrenheit is % lf . \n" , 36.0, c_to_f (36.0));
    return 0;
}
double c_to_f ( double c_temp )
{
    return 9.0 / 5.0 * c_temp + 32;
}
```

Function prototype

- A function prototype is something that tells you the function's name, parameters, and return type before the function is defined.
- A function prototype is the same as adding a semicolon(;) to the function header. However, in a function prototype, you don't need to write the parameter names. You only need to write the parameter data types.

```
double c_to_f(double);  
int get_integer(void);
```

The parameter name may be omitted.
You must add ; at the end.



Function definitions 1

```
int NumberCompare(int num1, int num2)
{
    if(num1>num2)
        return num1;
    else
        return num2;
}

int main(void)
{
    printf("The larger number between 3 and 4 is %d. \n", NumberCompare(3, 4));
    printf("The larger number between 7 and 2 is %d. \n", NumberCompare(7, 2));
    return 0;
}
```

Return statements can be placed in the middle as many times as needed.

Execution results

The larger number between 3 and 4 is 4.

The larger number between 7 and 2 is 7.

```
printf("The larger number between 3 and 4 is %d. \n", NumberCompare(3, 4));
printf("The larger number between 7 and 2 is %d. \n", NumberCompare(7, 2));
```

```
printf("The larger number between 3 and 4 is %d. \n", 4);
printf("The larger number between 7 and 2 is %d. \n", 7);
```

After calling the NumberCompare function in the two sentences above, it becomes as shown on the left.

Function definitions 2

ch10_func.c

```
int AbsoCompare(int num1, int num2); // Returns the integer with the largest absolute value
int GetAbsoValue(int num); // Returns the absolute value of the argument

int main(void)
{
    int num1, num2;
    printf("Enter two integers: ");
    scanf("%d %d", &num1, &num2);
    printf("The integer with the larger absolute value between %d and %d: %d\n",
           num1, num2, AbsoCompare(num1, num2));
    return 0;
}

int AbsoCompare(int num1, int num2)
{
    if(GetAbsoValue(num1) > GetAbsoValue(num2))
        return num1;
    else
        return num2;
}

int GetAbsoValue(int num)
{
    if(num < 0)
        return num * (-1);
    else
        return num;
}
```

```
if(GetAbsoValue(num1) > GetAbsoValue(num2))
    . . . .

if( 5 > 9 ) After calling the GetAbsoValue
    . . . .
```

As this example shows, the function call can be placed anywhere.

Execution results

Input two integers: 5-9

The integer with the larger absolute value between 5 and -9: -9

Parameters and return values

Syntax

Structure of a function

Return type Function name Parameters

`int max(int x, int y)`

{

`if( x > y )`

`return x;`

`else`

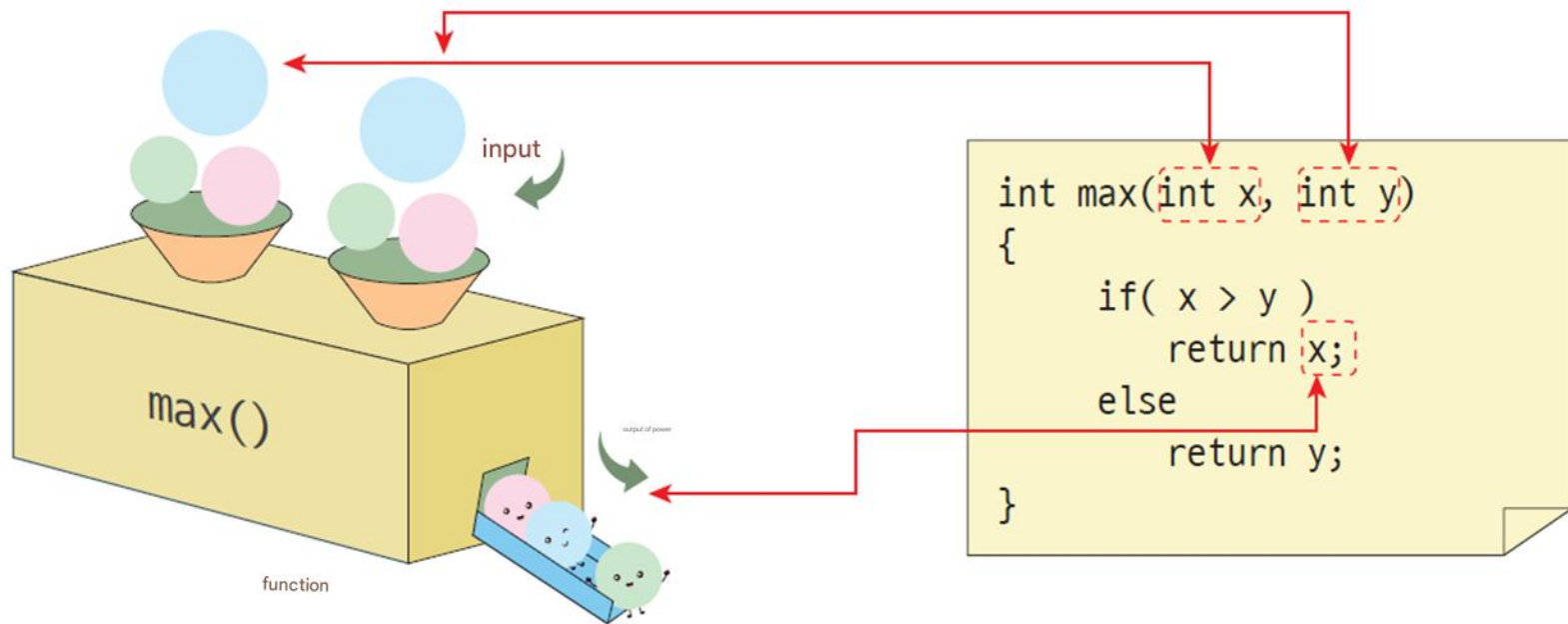
`return y;`

}

Function body

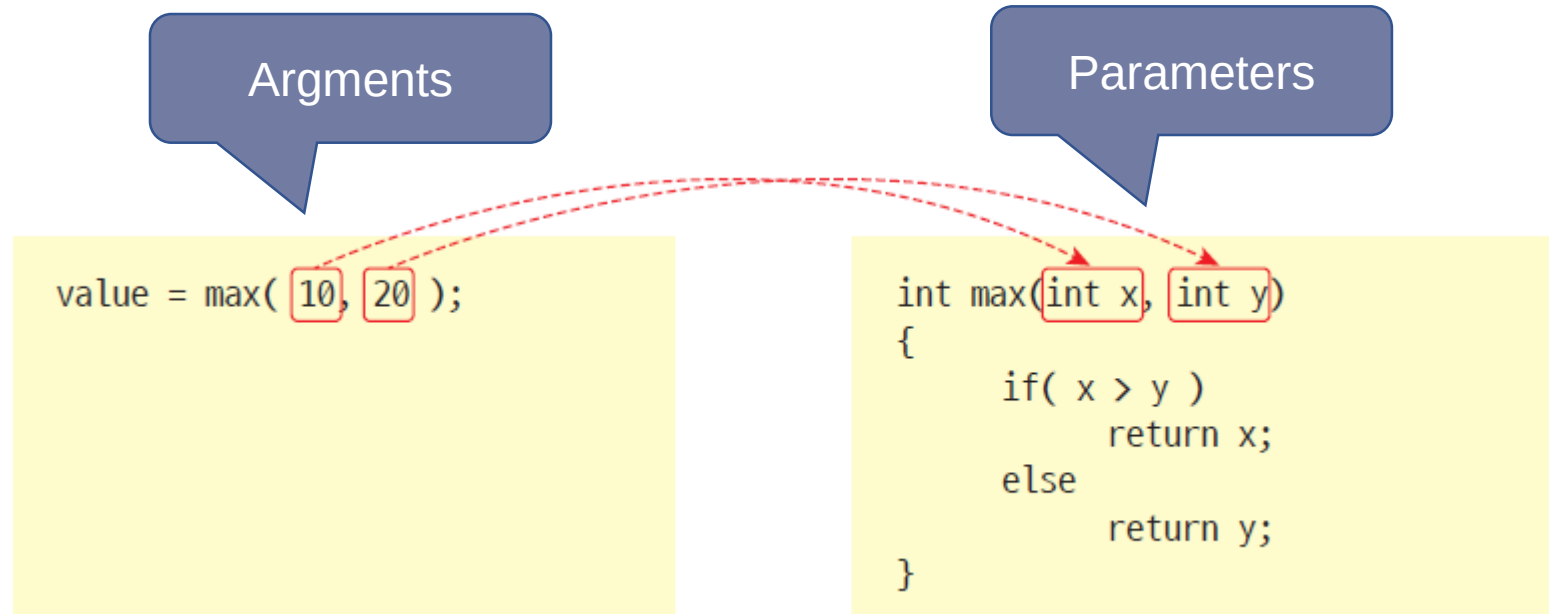


Parameters and return values



Arguments and parameters

- An argument is a value actually passed to a function by the calling program.
- A parameter is a variable that receives this value.



Arguments and parameters

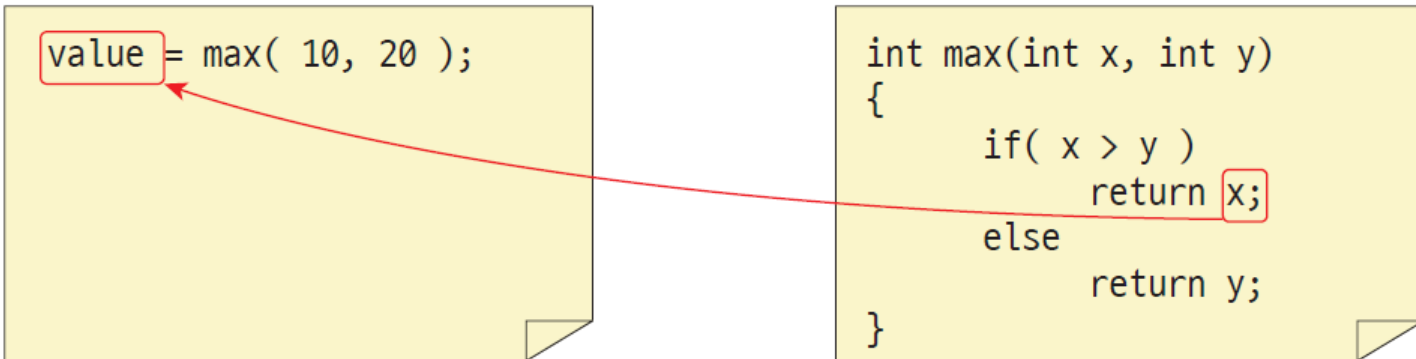
- If there are no parameters, you can write void in the parameter position, like `print_stars(void)`, or write nothing, like `print_stars()`
- The arguments may vary each time the function is called. (...)
- The number of parameters must match exactly. If the number of parameters and the number of arguments do not match, an error will occur that is very difficult to find.

```
max(10);           // When calling the max() function, there must be two arguments.  
max( );           // When calling the max() function, there must be two arguments.
```



Return value

- The return value is the result of the operation that the function returns to the place where it called.
- To return a value, write a formula after the return statement and the value of the formula will be returned.
- There can be **multiple arguments, but only one return value.**
(possible to return multiple values with pointer-parameter or structure)



C programming



Chapter 09-2. The existence period of variables and
Access Scope 1: Local Variables

C Programming:

C programming

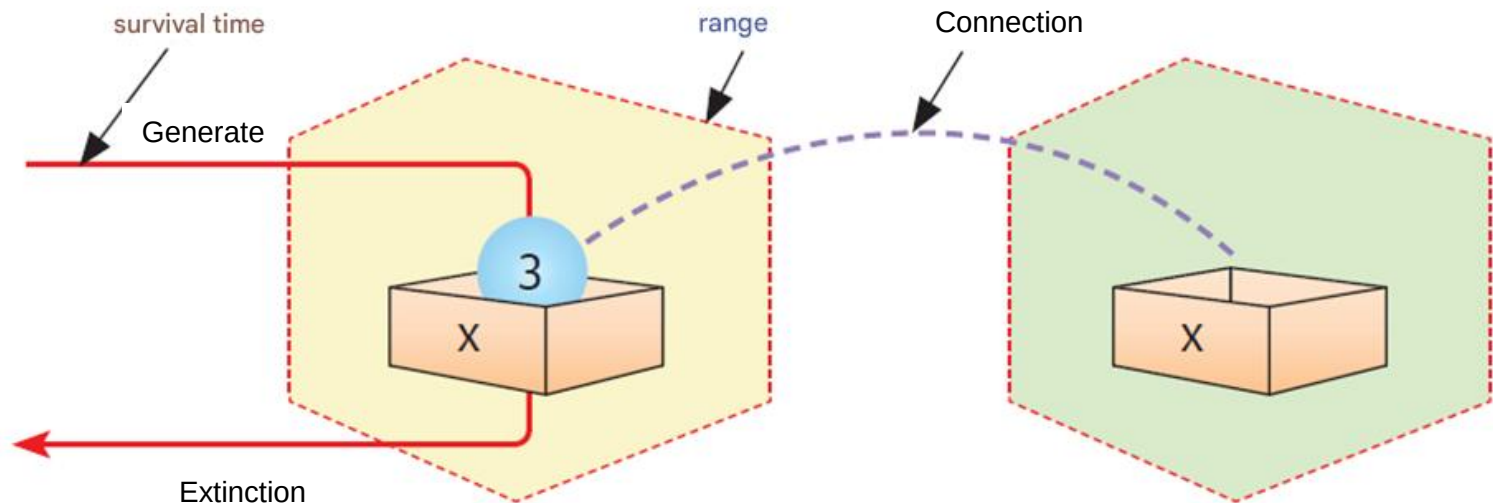


Chapter 09-3. Global variables , static variables ,
and register variables

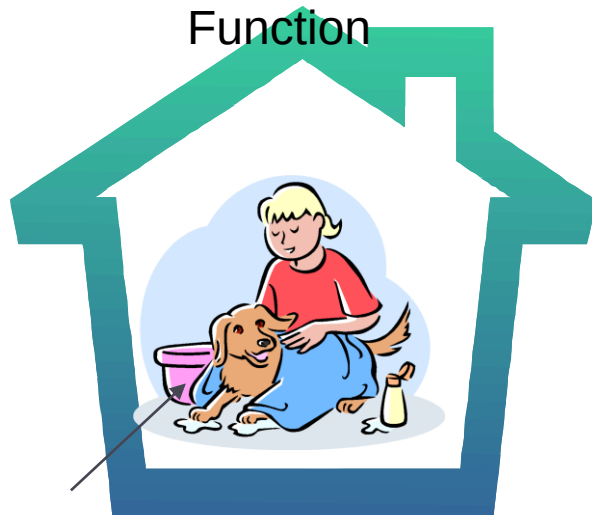
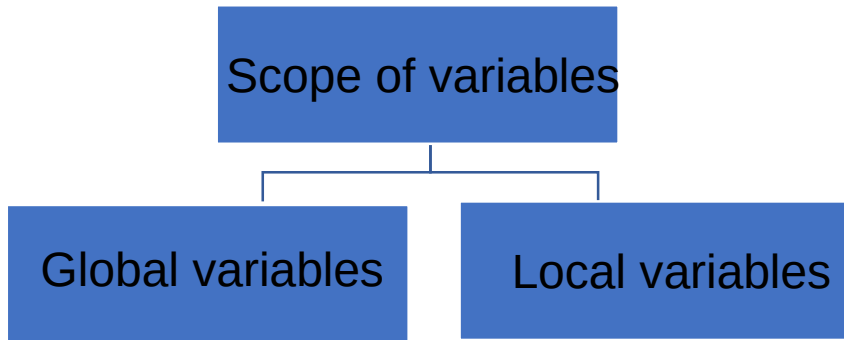
C Programming:

Variable properties

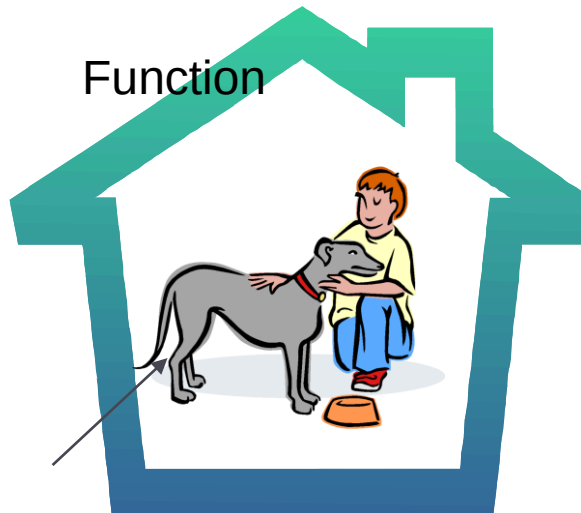
- **Variable properties : name, type, size, scope, lifetime, linking**
 - ✦ **Scope** : The scope in which a variable is available, its visibility
 - ✦ **Lifetime** : The time it exists in memory
 - ✦ **Linkage** : Status of connection with variables in other areas



Scope of variables



Local variables

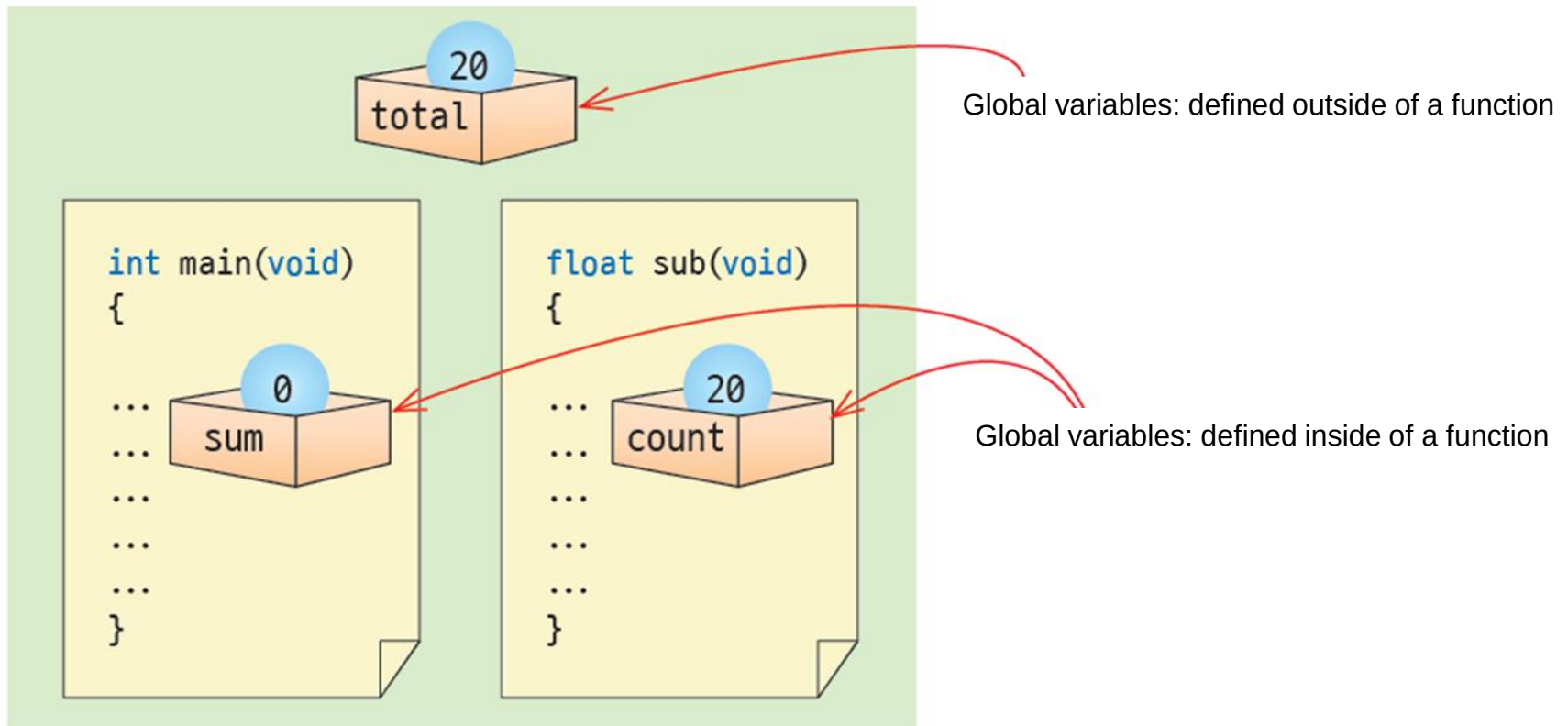


Local variables

Global variables



Global Variables and local variables



Local variables

- A local variable is a variable declared within a block.

```
int sub(void)
```

```
{
```

```
    int x = 0;
```

```
    while(flag!= 0){
```

```
        int y;
```

```
        ...
```

```
    }
```

```
    y = 0; // Error!!
```

```
    ...
```

```
}
```

The scope in which local variable x can be used

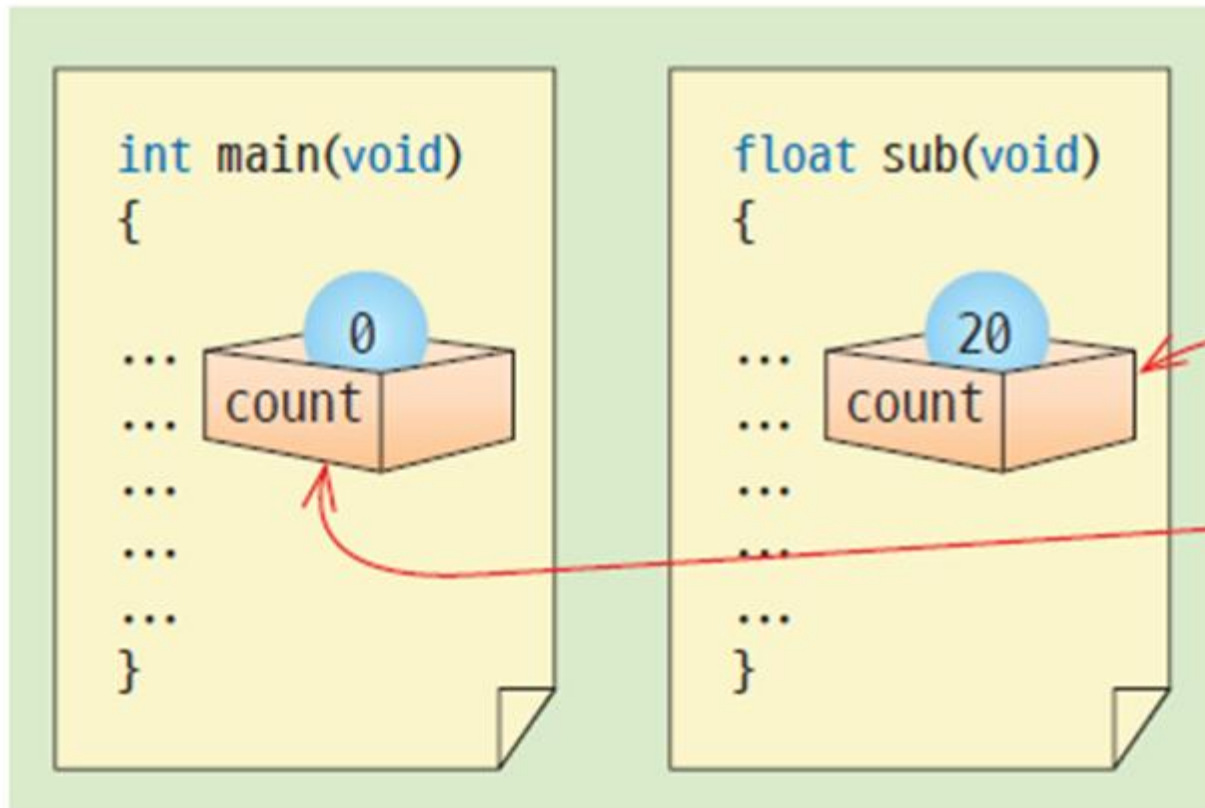
The scope in which global variable x can be used

Error because y was used outside the block where it was declared

Local variables must not leave the block in which they are declared.



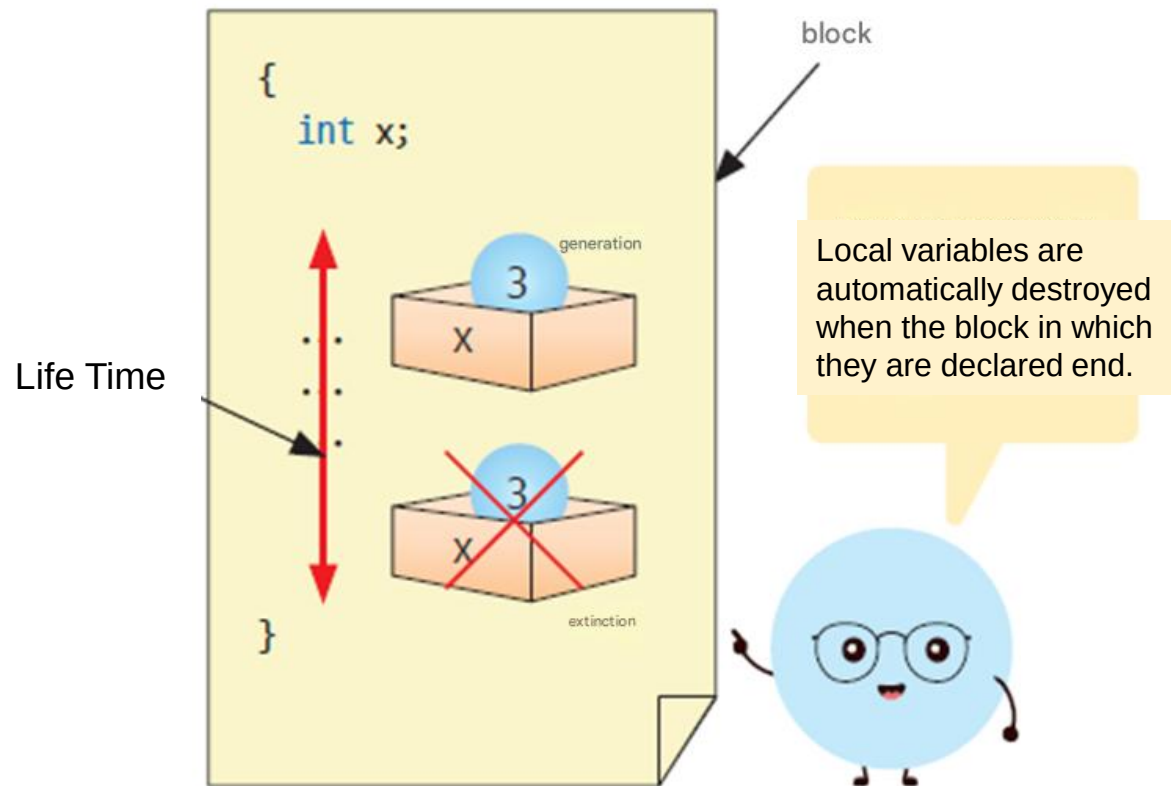
Local variables with the same name



The names can be the same as long as the blocks are different.



Life time of local variables



Function parameters

- Parameters defined in the header part of a function are also a type of local variable. That is, they have all the characteristics of local variables.
- What makes it different from local variables is that they are initialized with the argument values when the function is called.

```
int inc ( int counter )  
{  
    counter++;  
    return counter;  
}
```

Parameters are also a kind of local variable



Local variables that exist and are accessible only within a function

```
int SimpleFuncOne(void)
{
    int num = 10; // SimpleFuncOne's num is valid from this point onwards
    num++;
    printf("SimpleFuncOne num: %d \n", num);
    return 0;    // The last valid sentence for SimpleFuncOne's num
}

int SimpleFuncTwo(void)
{
    int num1=20;    // num1 is valid from now on
    int num2=30;    // num2 is valid from now on
    num1++, num2--;
    printf("num1 & num2: %d %d \n", num1, num2);
    return 0;    // num1, num2 last valid sentence
}

int main(void)
{
    int num=17;    // From now on, main's num is valid
    SimpleFuncOne();
    SimpleFuncTwo();
    printf("main num: %d \n", num);
    return 0;    // The last statement where main's num is valid
}
```

Variables declared within a function are called local variables.

Local variables can only be accessed within a function after they are declared.

It is not possible to declare variables with the same name within a region(function).

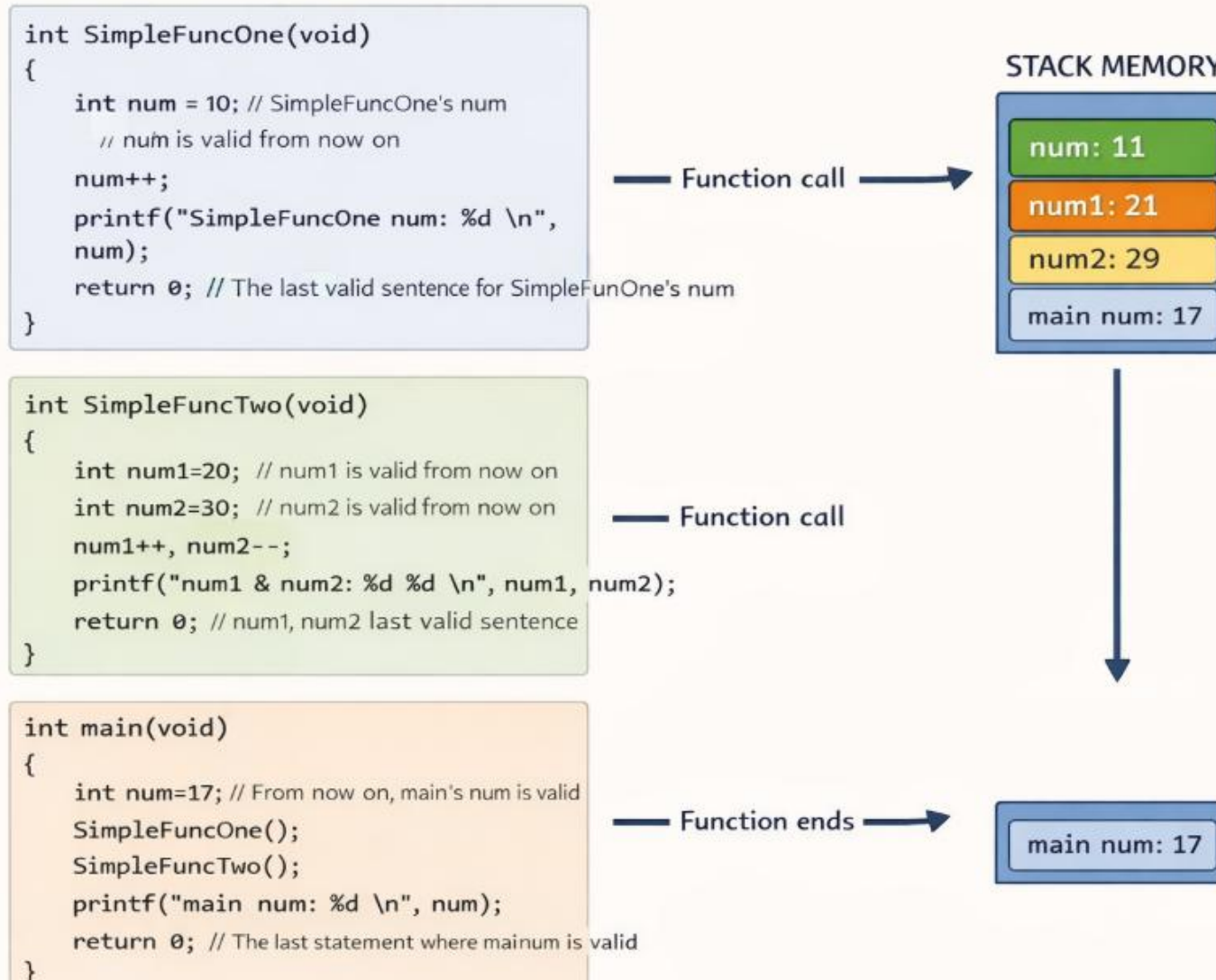
It is possible to declare variables with the same name in different regions.

When you leave that area, the local variables are destroyed and are allocated each time they are called.

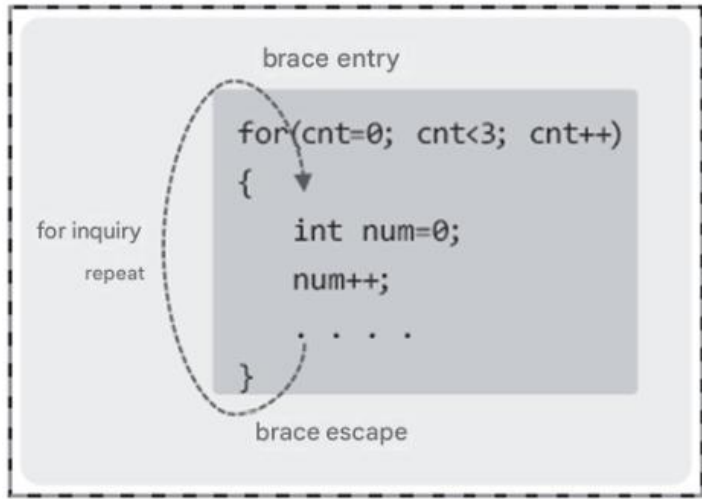
Execution results

```
SimpleFuncOne num: 11
num1 & num2: 21 29
main num: 17
```

Observing the allocation and destruction of memory space



Various types of local variables



Variables declared within the curly braces of a for loop are also local variables. And these local variables are destroyed when the for loop's curly braces are exited.

Therefore, local variables are allocated and destroyed as many times as the number of iterations of the for loop.

Local variables mask variables with the same name declared outside.

Execution results

Local variable num inside the if statement: 17
Local variable in main function num: 1

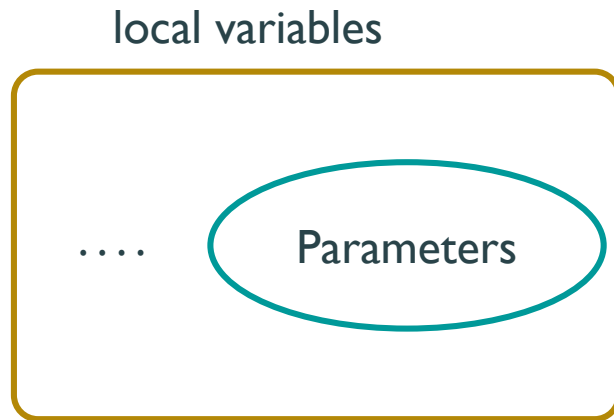
Execution result after commenting

Local variable num inside the if statement: 11
Local variable in main function num: 11

```
int main(void)
{
    int num=1;
    if(num==1)
    {
        int num=7; // Comment out this line and check the execution result!
        num+=10;
        printf("Local variable num inside if statement: %d \n", num);
    }
    printf("Local variable num in main function: %d \n", num);
    return 0;
}
```

The variable num declared within the if statement overrides the variable num in the main function .

Parameters, a type of local variable



Parameters are a type of local variable.

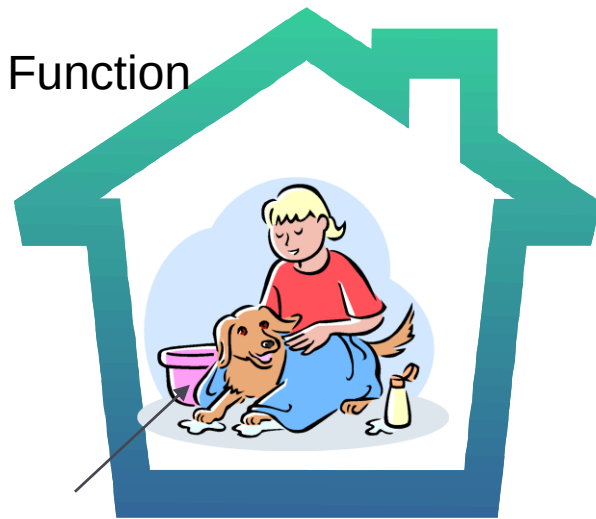
Parameters can only be accessed within the function in which they are declared.

When a declared function returns, the parameters are destroyed, just like local variables.

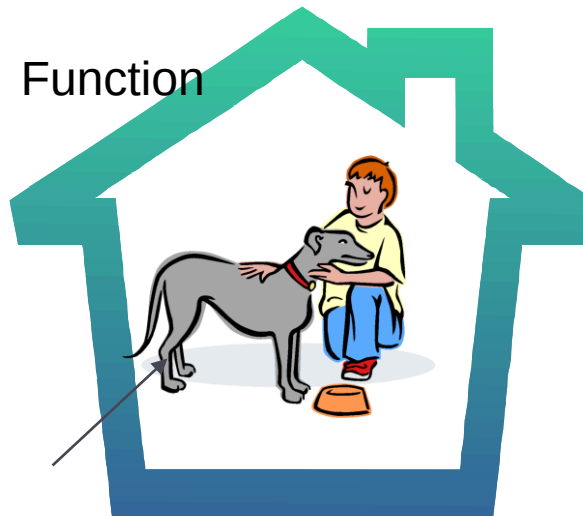


Global variables

- A global variable is a variable declared outside any function.
- The scope of a global variable is the entire source file.



Local variables



Local variables

Global variables



Initial values and life time of global variables

```
#include <stdio.h>
```

```
int A;
```

```
int B;
```

```
int add()
```

```
{
```

```
    return A + B;
```

```
}
```

```
int main( void )
```

```
{
```

```
    int answer;
```

```
    A = 5;
```

```
    B = 7;
```

```
    answer = add();
```

```
    printf ( " % d + % d = % d\n", A, B, answer);
```

```
    return 0;
```

```
}
```

Global variables
The initial value is 0

5 + 7 = 12

Scope of
global va
riables

Using global variables

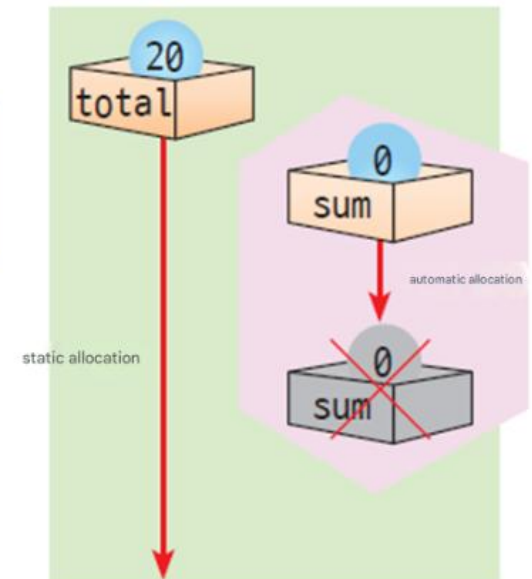
- ▶ Common data used by almost all functions should be made into global variables.
- ▶ Data used by only some functions should not be made into global variables, but passed as function arguments.



Survival period

- ▶ **static allocation**
 - ▶ Maintain throughout the program's execution time
- ▶ **automatic allocation**
 - ▶ Creation when entering a block
 - ▶ Destruction when exiting the block

Static allocation is a variable at runtime
It's there all the time, but automatic allocation is
It is destroyed when the block ends.



Understanding global variables and how to declare them

```
void Add(int val);
int num;    // Global variables are initialized to 0 by default
```

```
int main(void)
{
    printf("num: %d \n", num);
    Add(3);
    printf("num: %d \n", num);
    num++;    // Increment the value of global variable num by 1
    printf("num: %d \n", num);
    return 0;
}
```

```
void Add(int val)
{
    num += val;    // Increase the value of global variable num by val
}
```

```
num: 0
num: 3
num: 4
```

Execution results

The name of a local variable masks the name of a global variable.

Execution results

```
num: 12
num: 14
```

Global variables are declared outside of any function.

It is allocated in memory space when the program starts and exists until it ends.

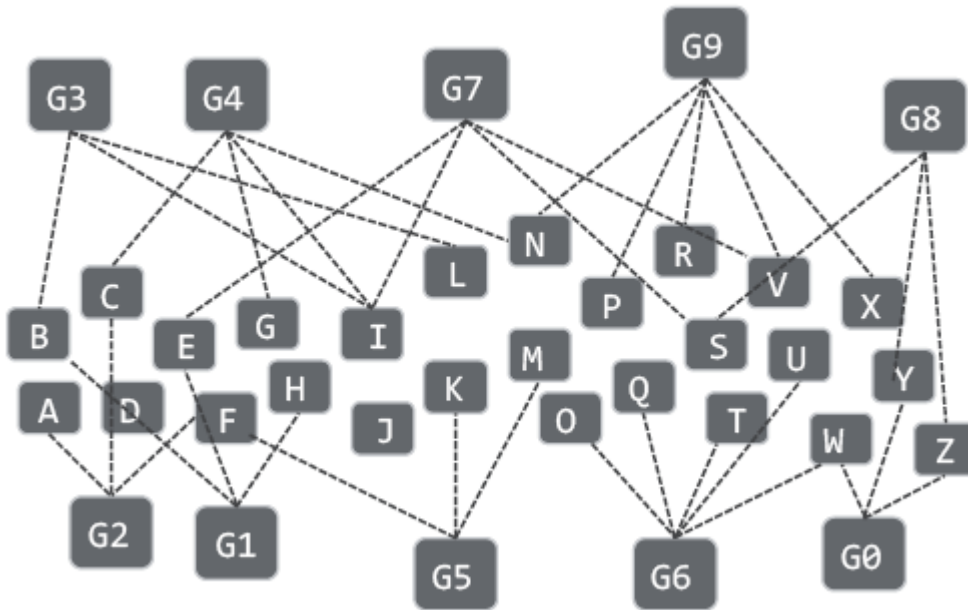
If not initialized to a separate value, it is initialized to 0.

Accessible from anywhere in the entire program.

```
int Add(int val);
int num=1;

int main(void)
{
    int num=5;
    printf("num: %d \n", Add(3));
    printf("num: %d \n", num+9);
    return 0;
}

int Add(int val)
{
    int num=9;
    num += val;
    return num;
}
```



Examples of access relationships between global variables and functions in G0~G9

Global variables! Overusing them is bad. Changing them can affect the entire program, and code dependent on them must be searched throughout the entire program.

Since they're accessible from anywhere ...

static variable with a static declaration added to a local variable

```
void SimpleFunc(void)
{
    static int num1=0; // If not initialized, initialize 0
    int num2=0;        // Initialize to garbage values if not initialized
    num1++, num2++;
    printf("static: %d, local: %d \n", num1, num2);
}

int main(void)
{
    int i;
    for(i=0; i<3; i++)
        SimpleFunc();
    return 0;
}
```

**Accessible only within the declared function.
(Local variable characteristics)**

**Only once and until the program ends.
It exists in the memory space.
(Global variable characteristics)**

*Execution
results*

```
static: 1, local: 1
static: 2, local: 1
static: 3, local: 1
```

“Actually, it **has the same characteristics as a global variable**. If not initialized, it is initialized to 0 like a global variable, and it is allocated and initialized at the same time as the program starts, and it remains in the memory space until the program ends! Then why was it declared in this location? It is to limit **the scope of access to SimpleFunc !**”

Static local variables are accessible only within a function, but their memory exists in the entire program area, not just the function (stack).



About static local variables

√ **One of the reasons why global variables are needed is as follows :**

The declared variable needs to continue to exist in the memory space even after the function exits .

√ **There are two ways to declare a variable that must continue to exist in memory space even after exiting the function .**

global variables, static local variables

√ **Static local variables are much more stable than global variables because their scope of access is much narrower.**

Let's minimize the declaration of global variables by using static local variables.



Faster ! register variable

```
int SoSimple(void)
{
    register int num=3;
    . . . .
}
```

register is a keyword that provides a hint.
The compiler may ignore it .

And because registers exist inside the CPU,
they are the fastest-access memory devices.

“I use this variable frequently, so storing it in the fastest-access register will help improve performance.”

Meaning of register variable declaration



[illegible]

C Programming:

What is recursion ?

← A function can also **call itself**. This is called recursion.

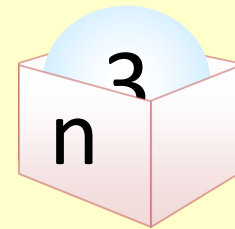
$$n! = \begin{cases} 1 & n=0 \\ n*(n-1)! & n \geq 1 \end{cases}$$



Calculating factorial

◀ Factorial Programming : Calculate the factorial of $(n-1)!$ by calling the function you are currently writing again (recursive call)

```
int factorial( int n)
{
    if ( n <= 1 ) return (1);
    else return (n * factorial(n-1) );
}
```



Structure of a factorial function

◀ The recursive algorithm consists of a part that recursively calls itself and a part that stops the recursive call.

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

```
int factorial(int n)
{
```

```
    if( n <= 1 ) return 1
```

```
    else return n * factorial(n-1);
```

```
}
```

part that stops circulation

The part that makes circular calls

Structure of printing 1~n function

The recursive algorithm consists of a part that recursively calls itself and a part that stops the recursive call.

1 2 3 4 5

```
#include <stdio.h>
```

```
void print_numbers(int n) {
    if (n == 0)
        return;    // base case (recursive end)

    print_numbers(n - 1); // recursive call
    printf("%d ", n);
}
```

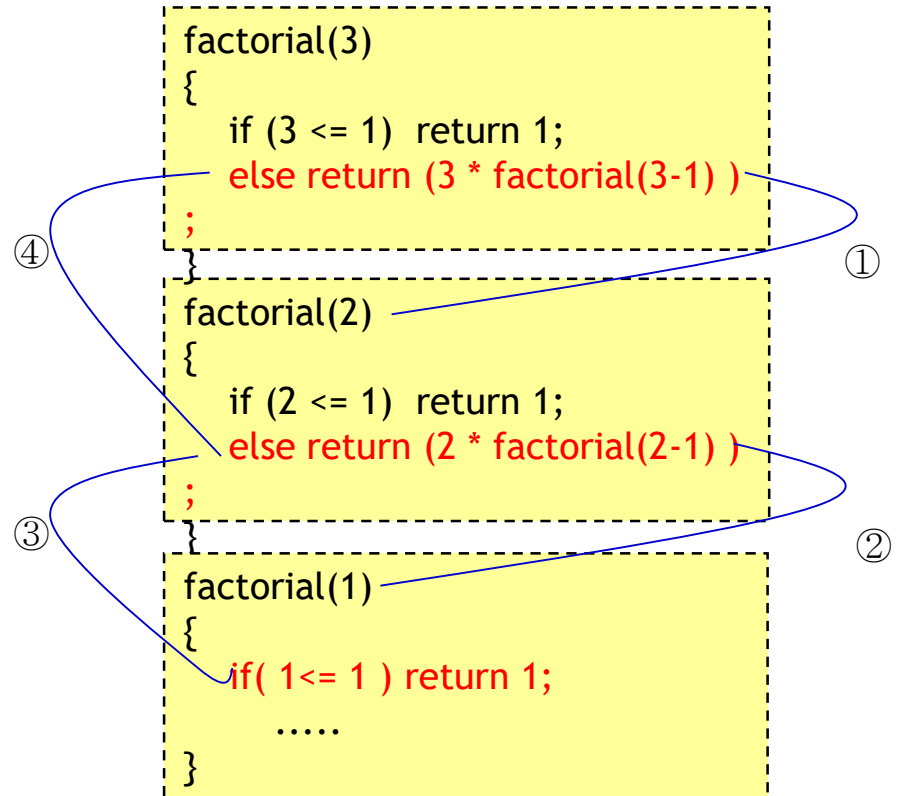
```
int main() {
    print_numbers(5);
    return 0;
}
```

```
print_numbers(5)
→ print_numbers(4)
→ print_numbers(3)
→ print_numbers(2)
→ print_numbers(1)
→ print_numbers(0)
→ return
← prints 1
← prints 2
← prints 3
← prints 4
← prints 5
```

Calculating factorial

◀ Factorial calling order

factorial(3)
= 3 * factorial(2)
= 3 * 2 * factorial(1)
= 3 * 2 * 1
= 3 * 2
= 6



Factorial calculation

```
// Calculate the
#include <stdio.h>

long factorial( int n )
{
    printf( "factorial(%d)\n" , n );

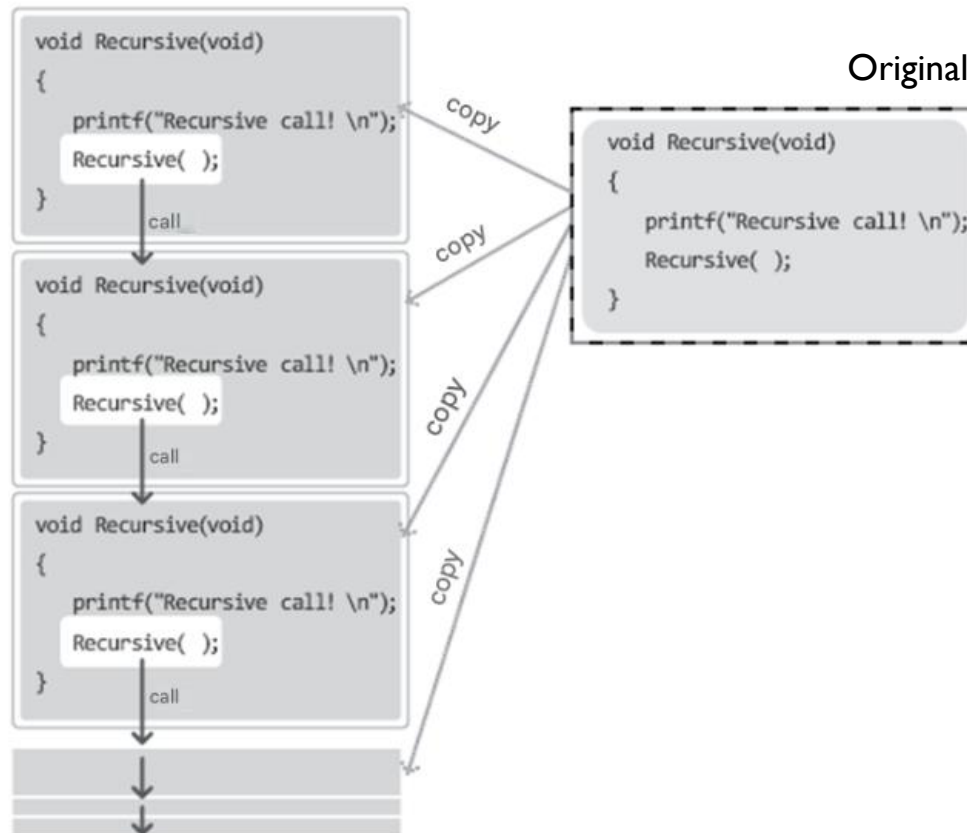
    if ( n <= 1)
        return 1;
    else
        return n * factorial( n - 1);
}

int main( void )
{
    int x = 0;
    long f;

    printf ( " Enter an integer :" );
    scanf ("%d", &x);
    printf ("%d! is %ld . \n", x, factorial(x));
    return 0;
}
```

Enter an integer : 5
factorial(5)
factorial(4)
factorial(3)
factorial(2)
factorial(1)
5 !

Basic understanding of recursive functions

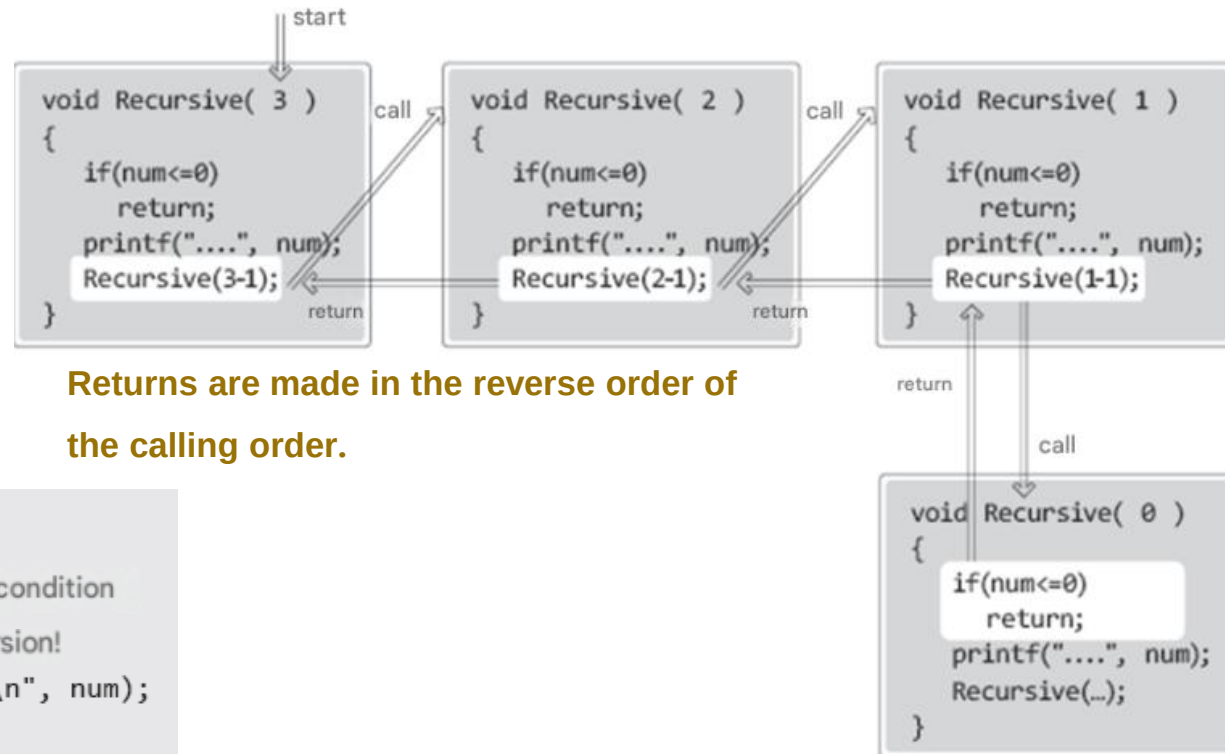


Understanding recursive function calls !

```
void Recursive(void)
{
    printf("Recursive call! \n");
    Recursive(); // 나! 자신을 재 호출한다.
}
```

A function defined in a way that calls itself is called a recursive function .

Example of a recursive function with an escape condition



Returns are made in the reverse order of the calling order.

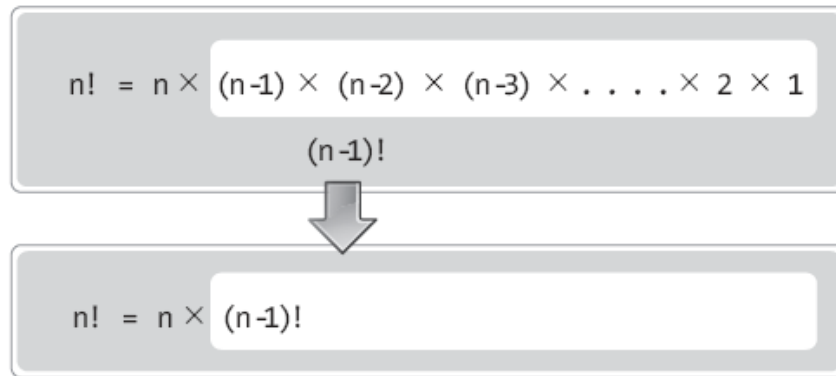
```
void Recursive(int num)
{
    if(num<=0)    // Recursion exit condition
        return; //Escape from recursion!
    printf("Recursive call! %d \n", num);
    Recursive(num-1);
}

int main(void)
{
    Recursive(3);
    return 0;
}
```

Execution results

```
Recursive call! 3
Recursive call! 2
Recursive call! 1
```

Recursive function design examples



Mathematical representation of factorial

$$f(n) = \begin{cases} n \times f(n-1) & \dots n \geq 1 \\ 1 & \dots n = 0 \end{cases}$$

Code implementation for $n \times f(n-1) \dots n \geq 1$

```
if(n>=1)
    return n * Factorial(n-1);
```

Code implementation for $f(n)=1$

```
if(n==0)
    return 1;
```

```
if(n==0)
    return 1;
else
    return n * Factorial(n-1);
```

Example of factorial function

```
int Factorial(int n)
{
    if(n==0)
        return 1;
    else
        return n * Factorial(n-1);
}

int main(void)
{
    printf("1! = %d \n", Factorial(1));
    printf("2! = %d \n", Factorial(2));
    printf("3! = %d \n", Factorial(3));
    printf("4! = %d \n", Factorial(4));
    printf("9! = %d \n", Factorial(9));
    return 0;
}
```

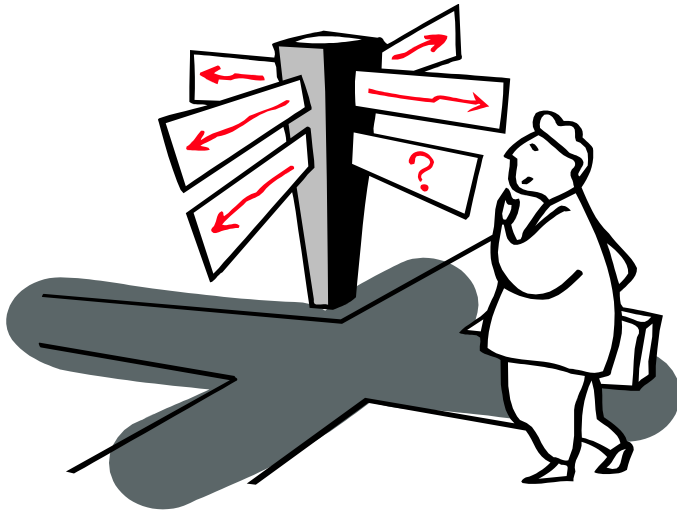
C language supports recursive function calls means that the range of expressions is wide!

C language, you can directly translate a recursively written expression into code.

Execution results

```
1! = 1
2! = 2
3! = 6
4! = 24
9! = 362880
```





Chapter 09 That's it . Do you have any questions ?

Random number function

- A random number is a number that is generated randomly without any pattern .
- Random numbers It is essential in cryptography, simulations, games, etc.
- rand()
 - ↳ Function to generate random numbers
 - ↳ Generate a random number from 0 to RAND_MAX



Random number function

`rand()`:

- Returns a pseudo-random number between 0 and `RAND_MAX` (a large constant, like 32767 or higher).
- Each call gives the next number in the sequence.
- If you don't call `srand()`, the sequence will always be the same when you run the program.

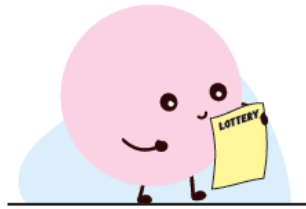
`srand(unsigned int seed)`:

- Seeds the random number generator with a starting point (`seed`).
 - Using the same seed always gives the same sequence of `rand()` numbers.
 - Common usage: `srand(time(NULL));` to seed with the current time, so you get different random numbers each time you run.
-



Example : Generating Lotto Numbers

◀let's write a program to generate lottery numbers. Lotto numbers are made up of six numbers from 1 to 45.

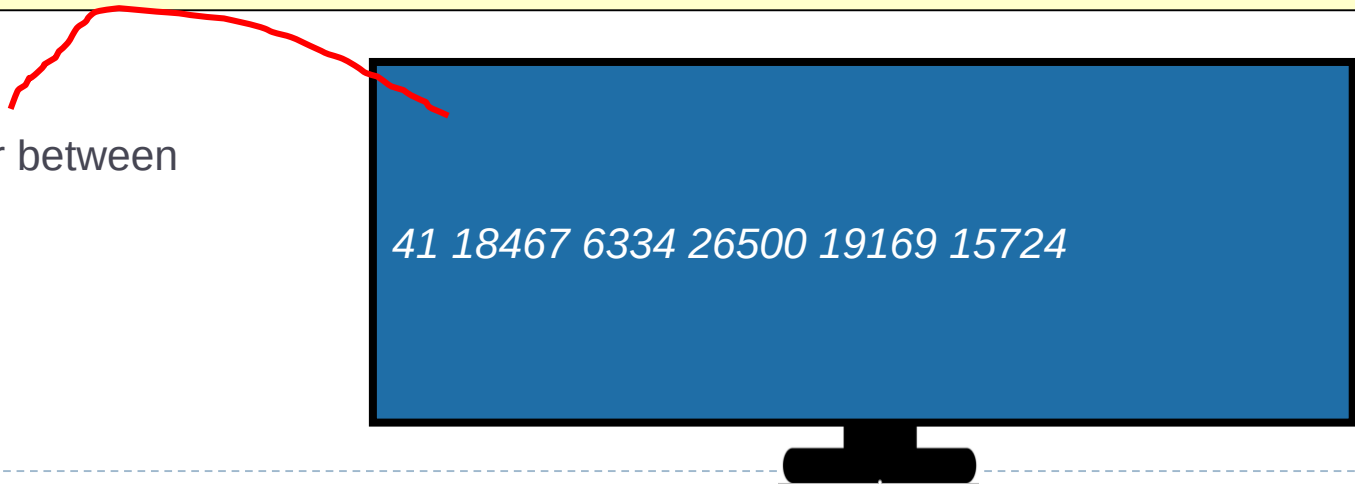


Practice Code

```
#include <stdio.h>
#include <stdlib.h>
int main( void )
{
    int i ;
    for ( i = 0; i < 6; i ++ )
        printf ( "%d ", rand());

    return 0;
}
```

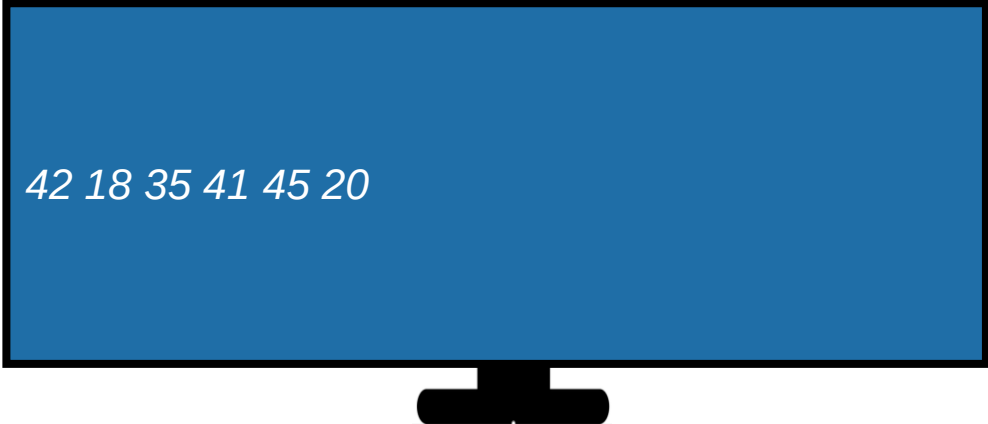
Generate an integer between
0 and 32767



41 18467 6334 26500 19169 15724

Limited to 1 to 45

```
←printf ("%d ", 1+(rand()%45));
```



42 18 35 41 45 20

←But every time I run it, the same random number is always generated .



Practice Code

```
#include <stdio.h>
#include <stdlib.h>
int main( void )
{
    int i ;
    srand(time(NULL)); // initialize random seed !
    for ( i = 0; i < 6; i ++ ) {
        printf ( "%d " , rand() % 45 + 1);
    }
    return 0;
}
```

Generate an integer between
1 and 45

